

Unit 4: Introduction to Templates and Views

Unit Objective: *In this unit, we will build a Personal Website containing a Homepage and an About page while learning more about Django's templates, function-based views, and testing. Templates are the presentation layer that controls how data is displayed, but they also allow for inheritance and basic logic consistent with Django's design philosophy of Don't Repeat Yourself (DRY). Views combine URLs and templates while adding quite a bit of logic to the data; they are often considered the logic layer in a Django app. The third fundamental concept is testing, which is vital to any web project and well-supported in Django. By the end of this chapter, you will have learned how to work with simple function based views and templates and write your first tests.*

Homepage

A Django "view" is a Python function that accepts a Web request and returns a Web response. When a Web page is requested, Django automatically creates an HttpRequest object with metadata about the request. The view returns an HttpResponse object.

We will begin by making some changes to our existing pages app to create a homepage using just a view and a URL dispatcher. Then, we will create an About page using a template and more complex view logic.

For our home page, we will retain most of the code inside pages/views.py. See the changes below indicated in a yellow highlight:

```
# pages/views.py
from django.http import HttpResponse
from django.shortcuts import render

def home_page_view(request):
    return HttpResponse("Homepage")
```

The next step is reviewing the urls.py file within the pages app that imports path from django.urls, home_page_view from the views.py file in the local directory, and sets a route at the empty string, "", that calls home_page_view. As can be seen there are no changes to the pages/urls.py file.

```
# pages/urls.py
from django.urls import path
from .views import home_page_view
urlpatterns = [
    path("", home_page_view),
]
```

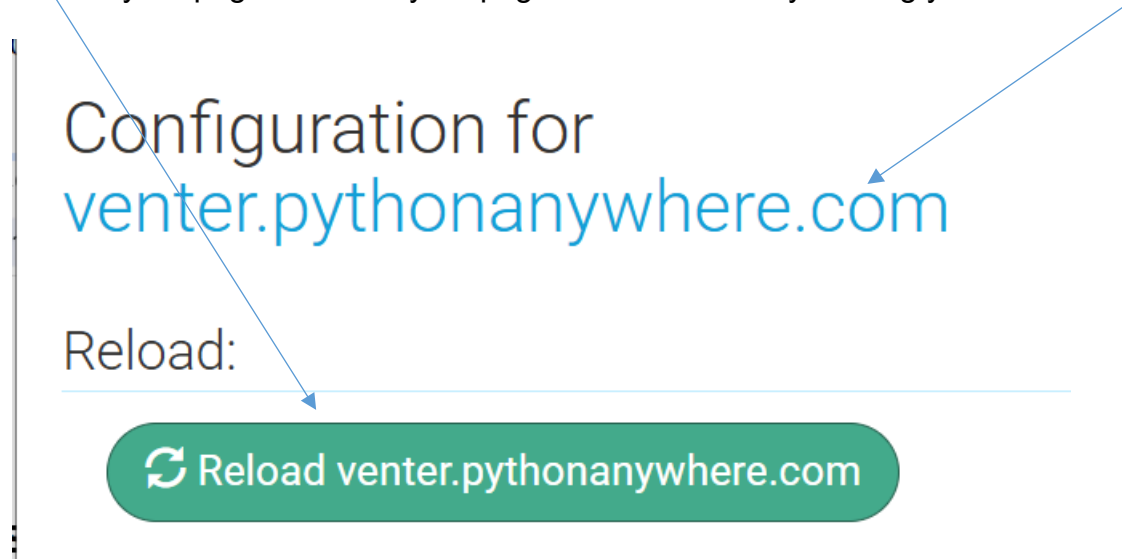
Also review the project-level `urls.py` file, the initial entry point for all URL requests. We import the `include` function to include other URL configurations and set a URL route of `""` for the `pages` application. There should also be now changes to this file:

```
# django_projects/mysite/mysite/urls.py

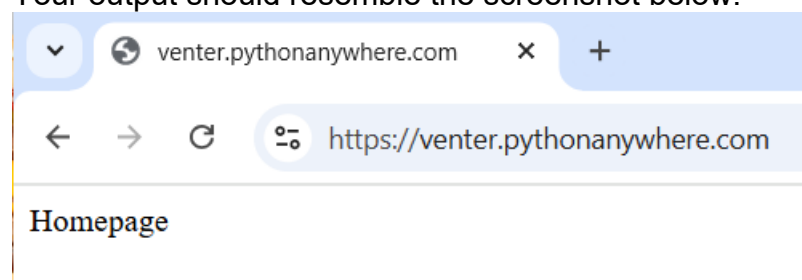
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("pages.urls")), # new
]
```

Reload your page and view your page in the browser by clicking your webserver link:



Your output should resemble the screenshot below:



Function-Based View About Page

`render()` is a Django shortcut function useful for working with a template. The first parameter in `render()` is the request object, and the second parameter is a template name. That means if we want to create a view for an about page using a template called `about.html` we can write it as indicated in the yellow highlighted code below. Also add this code to your `pages/views.py` file.

```
# pages/views.py
from django.http import HttpResponse
from django.shortcuts import render

def home_page_view(request):
    return HttpResponse("Homepage")

def about_page_view(request): # new
    return render(request, "pages/about.html")
```

The new view is named `about_page_view` and has as its first parameter the `HttpRequest` object, which we've named `request`. It uses the `render()` function to return the request object and a related template named `"pages/about.html"`.

Templates

Templates are the presentation layer in Django, providing a convenient way to generate HTML files that can *also* include CSS, JavaScript, and media such as images. By default, Django's template loading engine looks for a `"templates"` subdirectory in each app. That means we *could* add a `templates` directory within the `pages` app and then include the `about.html` in the following way:

```
└─ pages
   └─ templates
      └─ about.html
```

This approach works, but it is not a best practice. Django chooses the first template it finds whose name matches. What happens if there are `about.html` files within two separate apps? Django cannot easily tell which one should be used in our function. Fortunately, a simple fix is to *namespace* template files by placing them within another directory containing the application name. In other words: `app -> templates -> app -> template file`. In our `pages` app, that would look like this:

```
└─ pages
   └─ templates
      └─ pages
         └─ about.html
```

As a Django best practice, you should always adopt this approach when storing template files within an app. Create these new directories within your text editor or directly go to your bash shell and execute the commands on the command line:

```
(.ve52) $ cd~
(.ve52) $ mkdir django_projects/mysite/pages/templates
(.ve52) $ mkdir django_projects/mysite/pages/templates/pages
```

Run the following commands to verify that you have created to correct file structure:

```
(.ve52) $ cd~
(.ve52) $ tree
```

Your directory structure for the pages folder should look like the screenshot below:

```
└─ pages
   └─ __init__.py
   └─ __pycache__
      └─ __init__.cpython-313.pyc
      └─ admin.cpython-313.pyc
      └─ apps.cpython-313.pyc
      └─ models.cpython-313.pyc
      └─ urls.cpython-313.pyc
      └─ views.cpython-313.pyc
   └─ admin.py
   └─ apps.py
   └─ migrations
      └─ __init__.py
   └─ models.py
   └─ templates
      └─ pages
   └─ tests.py
   └─ urls.py
   └─ views.py
```

Then, add a new file called `about.html` within the `pages/templates/pages` directory.

`/home/venter/django_projects/mysite/pages/templates/`  `pages`
14% full – 73.9 MB of your 512.0 MB quota [More Info](#)
[Open Bash console here](#)

Directories

New directory

Files

New file

The `about.html` file will have a `<h1>` tag for a headline and a `<p>` tag for paragraph text. Add the following code to your `about.html` file.

```
<!-- pages/templates/pages/about.html -->
<h1>About page</h1>
<p>This is the new template-powered About page.</p>
```

`/home/venter/django_projects/mysite/pages/templates/pages/about.html`

 Keyboard shortcuts: [Share](#) [Save](#) [Save as...](#) [Refresh](#)

```
<h1>About page</h1>
<p>This is the new template-powered About page.</p>
```

Our template is complete! The view, `about_page_view`, can access the template when called, but the remaining step is configuring a URL dispatcher.

URL Dispatcher

At this point, the pattern for adding a new URL route should start to feel familiar. We import our view, `about_page_view`, and then set its path, `about/`, and the view name.

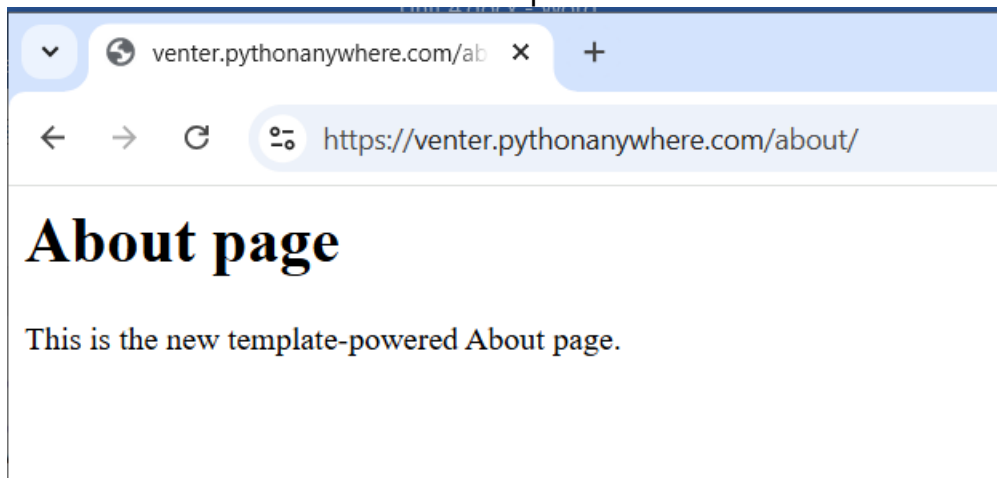
```
# pages/urls.py
from django.urls import path
from .views import home_page_view, about_page_view # new

urlpatterns = [
    path("about/", about_page_view), # new
    path("", home_page_view),
]
```

And that's it! Start up the development web server by clicking on reload in the Web tab and clicking the link to your webserver url.

Navigate to <https://studentnumber.pythonanywhere.com/about>

Your screen should resemble the output below:



The Django Template Language

All web frameworks (including Django) need a way to generate HTML dynamically, and the most common approach is to use templates that contain static content and a special syntax for inserting dynamic content. Django has its own [templating language](#) that is not attempting to replace dedicated JavaScript front-ends like Vue, React, or Angular. Rather, it is a deliberately limited—but still powerful—way to use comments, variables, filters, tags, and template inheritance. Let's revisit our about.html template to try out some of these features. We'll begin with [comment](#) tags that allow us to add code that is readable to the developer but not displayed on the page. Anything between `{% comment %}` and `{% endcomment %}` tags will be ignored.

Add the template code highlighted below to your about.html page:

```
<!-- pages/templates/pages/about.html -->
<h1>About page</h1>
<p>This is the new template-powered About page.</p>
{% comment %}This is just a comment. It won't appear on the web page!{%
endcomment %}
```

If you save the file and refresh the web browser, the comment will not be visible. It is not rendered on the template HTML sent to the about_page_view and then the web browser.

Template Context

Whenever a Django template is rendered, a [context](#) is also created. This context is a dictionary-like object with variable names for keys and variable values as values. We can and will update this context with information, often from a database. When you render a template with a given context, every key in the context dictionary becomes a variable in the template that you can access and use. The [render\(\)](#) shortcut function expects parameters in the following order:

1. The HttpRequest object conventionally named request
2. The template_name
3. The context dictionary

In other words: `render(request, template_name, context)`. By default, context is set to None, but we can manually add information to see it in action.

Make the changes highlighted in yellow to your views.py file

```
#django_projects/mysite/pages/views.py

from django.http import HttpResponse
from django.shortcuts import render

def home_page_view(request):
    return HttpResponse("Homepage")

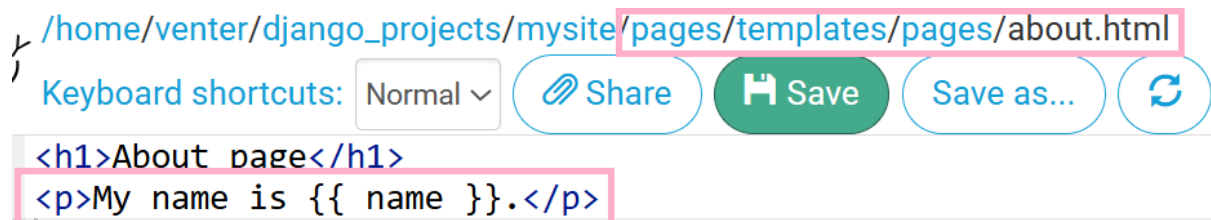
def about_page_view(request): # new
    context = {"name": "Alice"} # new
    return render(request, "pages/about.html", context) # new
```

Here, we've added a variable, context, containing a dictionary with the key variable of "name" and a value variable of "Alice." To display the context in our template, the syntax for **variables** is to surround the variable name with brackets {}. Since the key value is "name," we type {{ name }} to make it visible.

Change the content of your about.html file to resemble the code below:

```
<!-- pages/templates/pages/about.html -->
<h1>About page</h1>
<p>My name is {{ name }}.</p>
```

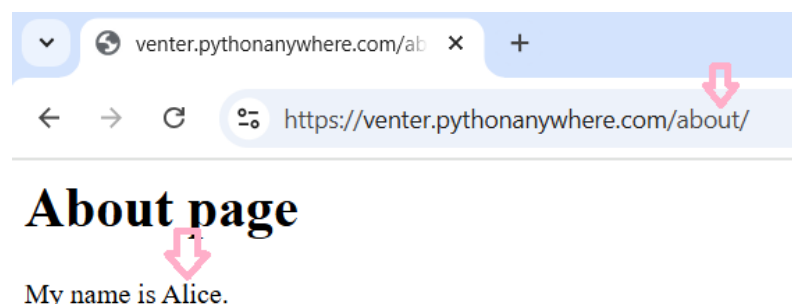
In pythonanywhere your screen should resemble the below screenshot.



Start up the development web server by clicking on reload in the Web tab and clicking the link to your webserver url.

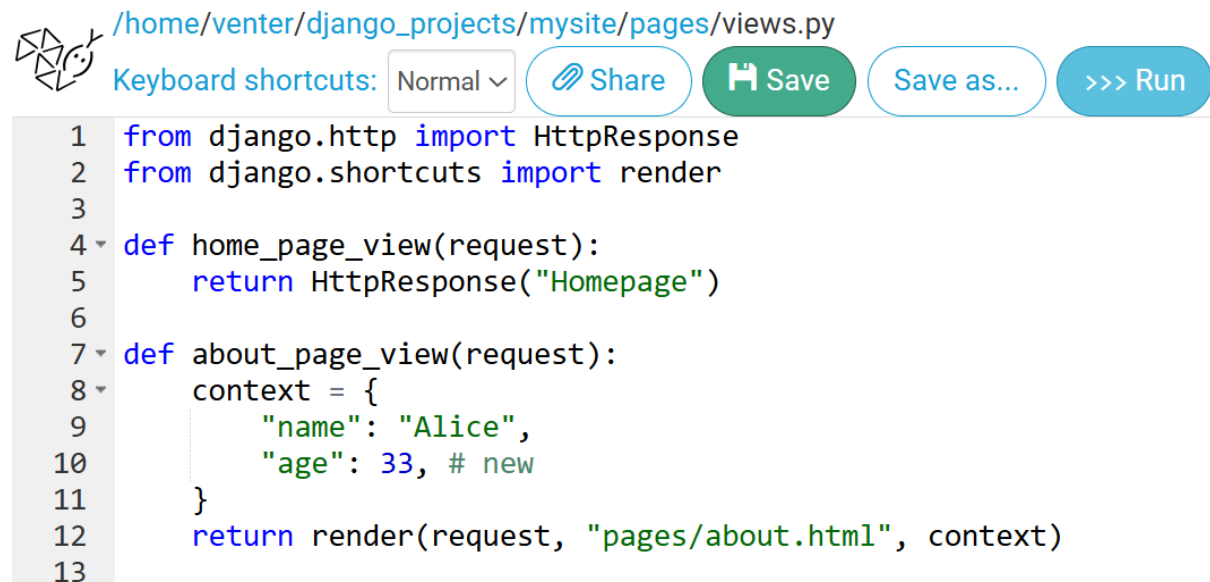
Navigate to <https://studentnumber.pythonanywhere.com/about>

Your screen should resemble the output below:



We can also add other key/value pairs to the context. For example, let's add an "age."

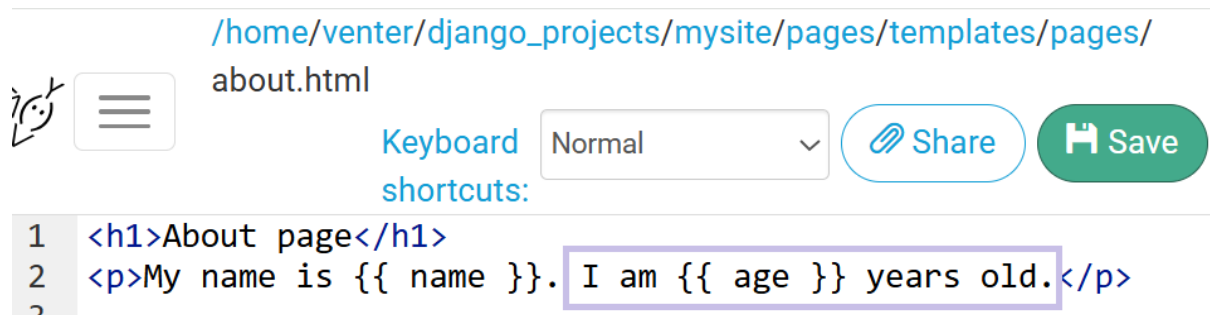
Add code for the age key/value pair of your views.py file as shown in the screenshot below:



```
/home/venter/django_projects/mysite/pages/views.py
Keyboard shortcuts: Normal ▾ Share Save Save as... >>> Run
1 from django.http import HttpResponse
2 from django.shortcuts import render
3
4 def home_page_view(request):
5     return HttpResponse("Homepage")
6
7 def about_page_view(request):
8     context = {
9         "name": "Alice",
10        "age": 33, # new
11    }
12    return render(request, "pages/about.html", context)
13
```

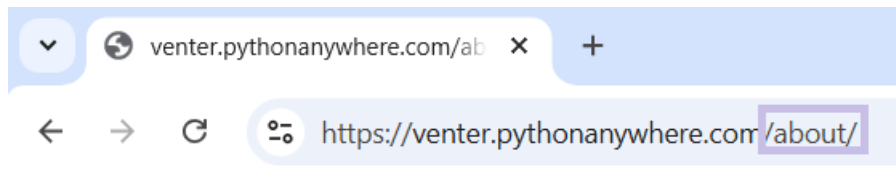
To display the age on our Web page, we can display it as a variable in the template using the syntax `{{ age }}`. Note that name is a string since it is in quotations, whereas 33 is an integer. Setting data types is important in Python code and Django applications, which we will explore shortly.

Make the changes to your about.html as shown below to also display the age on the about page.



```
/home/venter/django_projects/mysite/pages/templates/pages/
about.html
Keyboard shortcuts: Normal ▾ Share Save
1 <h1>About page</h1>
2 <p>My name is {{ name }}. I am {{ age }} years old.</p>
```

Reload and run your page again to see if your changes are displaying similar to the screenshot below.



About page

My name is Alice. I am 33 years old.